

Octro

Cahier des charges complet

Infrastructure de liquidité et de crédit prédictif

Model Predictive Control · Optimisation · Agents · XRPL

Version 2.0 — 12 septembre 2026
Kevin · Samet · Augustin

Spécification produit et technique pour le hackathon et le pilote B2B.

Sommaire

1	Comprendre Octro simplement	2
2	Les 5W du projet	3
3	Passage de FlowCredit à Octro	4
4	Ripple et différenciation stratégique	5
5	Cas d'usage mondiaux et modèle économique	6
6	MVP et limites de portée	7
7	Parcours produit et critères UX	8
8	Design humain et maquettage Pencil	9
9	Architecture et stack décidée	10
10	Utilisation de la VM existante	11
11	Modèle de données et invariants	12
12	Prévisions et jumeau de trésorerie	13
13	Formulation MPC et optimisation sous contraintes	14
14	Min cost flow CVaR et exemple calculé	15
15	Agents IA et outils MCP	16
16	Intégration XRPL et disponibilité réelle	17
17	DID et identité vérifiable	18
18	Stripe Identity conformité et données personnelles	19
19	API et contrats machine readable	20
20	États approbations et signatures	21
21	Sécurité et menaces	22
22	Résilience observabilité et exploitation	23
23	CI CD et organisation du dépôt	24
24	Plan de tests et Definition of Done	25
25	Répartition Kevin Samet Augustin	26
26	Roadmap hackathon et après	27
27	Démonstration indicateurs et décisions ouvertes	28
28	Sources vérifiées et registre documentaire	29

Statut : spécification proposée pour développement. Périmètre hackathon en environnement de test, puis pilote B2B encadré. Les exigences MUST sont obligatoires pour le périmètre indiqué ; SHOULD désigne une recommandation ; MAY une extension facultative. Les chiffres de simulation et les seuils ci-dessous sont des paramètres de conception, pas des performances observées.

1 Comprendre Octro simplement

Une entreprise peut avoir assez de revenus et manquer d'argent au moment où elle doit payer. Octro regarde les sommes disponibles, les paiements attendus et les échéances. Il anticipe le manque, compare des financements autorisés et propose une action avec son coût et ses risques. Après validation et signature, il suit le financement sur XRPL. Lorsque l'argent arrive réellement, il prépare le remboursement selon le contrat.

Exemple pédagogique : un opérateur dispose de 20 000 unités monétaires, doit payer 100 000 maintenant et attend 110 000 dans 36 heures. Pour conserver une réserve de 10 000, il lui manque 90 000. Octro peut proposer ce montant si un prêteur l'autorise, si les fonds sont disponibles et si le remboursement reste crédible en cas de retard. Une prévision n'est jamais un encaissement.

Octro est une infrastructure mondiale de liquidité et de crédit prédictif : un moteur utilisable par API et une interface de trésorerie. L'entreprise garde la décision ; le prêteur garde l'underwriting ; le moteur calcule ; les agents coordonnent et expliquent ; XRPL porte les opérations compatibles. Octro ne crée ni liquidité ni garantie de remboursement.

Le premier produit cible les décalages de règlement des PSP et marketplaces B2B. L'ambition mondiale concerne l'architecture et les connecteurs ; chaque ouverture commerciale exige des partenaires, des règles et des rails locaux validés.

2 Les 5W du projet

WHAT — Une boucle de contrôle prédictif de trésorerie. Entrées : soldes confirmés, événements économiques, prévisions, dettes, limites, offres de prêteurs et disponibilité des vaults. Sorties : manque daté, plan de financement, scénarios de risque, justification, dossier d’approbation et suivi du prêt. Le produit comprend dashboard, API, moteur quantitatif, orchestration d’agents, outils MCP et adaptateur XRPL.

WHY — Réduire les paiements manqués, les réserves excessives et le coût des financements de courte durée. Les soldes dispersés et les délais variables rendent une vue statique insuffisante. Le succès se mesure face à une politique de référence sur les mêmes données : coût total, déficit maximal, heures sous réserve, échéances respectées et temps opérationnel. Aucune économie garantie n’est affichée.

WHO — Le trésorier consulte et propose ; le signataire autorise ; le responsable risque définit les plafonds ; le responsable conformité vérifie les entités ; le prêteur ou broker accorde les termes ; le déposant fournit les capitaux ; l’intégrateur relie les systèmes. Kevin porte produit, UX et agents, Samet API et infrastructure, Augustin moteur financier et XRPL. Cette attribution décrit la responsabilité de livraison, pas des compétences présumées.

WHERE — Les données peuvent provenir de banques, PSP, ERP, mobile money et marketplaces. Le calcul et les dossiers restent hors chaîne ; seules les opérations et preuves minimales nécessaires utilisent XRPL. Le pilote commence dans une seule juridiction et un seul corridor ; les données sont hébergées dans une région contractuellement choisie. Le MVP utilise des données fictives et des actifs de test.

WHEN — Recalcul à l’arrivée d’un événement significatif et toutes les 15 minutes ; horizon initial 72 heures par pas d’une heure. Les événements proches restent traités à leur horodatage exact pour éviter qu’une agrégation masque un manque intrahoraire. Hackathon prévu sur une hypothèse de 48 heures à adapter au calendrier officiel ; pilote après validation technique, commerciale et juridique.

3 Passage de FlowCredit à Octro

Base analysée : FlowCredit_CDC_Complet_v1.1.pdf, 20 pages, extrait de l'archive v1.1. Ce CDC v2.0 remplace ses décisions de produit. L'ancien flux « activité → profil → offre » devient « état de trésorerie → prévision → optimisation → validation → financement → réconciliation → nouveau calcul ».

Les sections A et B de v1.1 sont réécrites autour du settlement gap B2B et du MPC. Le modèle EconomicEvent, la provenance, le consentement, les règles de prêteur et l'explicabilité sont conservés. Le scoring universel ne constitue plus le centre du produit.

Les sections C et D sont conservées et précisées : vault, broker, prêt, servicing, contrôle des signatures et audit ; React Native remplace la recommandation Next.js/Supabase. PostgreSQL devient la source financière ; MongoDB et S3 ont des rôles limités. Les sections E et K deviennent le périmètre, les critères de recette et la roadmap ci-dessous.

Les sections F à J sont recentrées : Octro Agent remplace FlowAgent ; orchestration bornée et MCP restent utiles ; Connector Factory, passeport financier grand public et divulgation sélective avancée passent après le MVP. Les garde-fous de v1.1 sur les doubles financements, données périmées et résultats réseau ambigus deviennent des invariants vérifiables.

Les promesses anciennes de portabilité mondiale ne valent pas autorisation de crédit. Le first-loss ne garantit pas le capital. Une créance attendue n'est pas automatiquement un collatéral on-chain. Le remboursement « automatique » dépend de fonds disponibles, d'un mandat et des règles exactes du prêt.

4 Ripple et différenciation stratégique

FAITS SOURCÉS — Ripple présente XLS-65/66 comme une infrastructure de crédit avec décision et risque hors chaîne, et exécution sur le ledger [S1, S2, S3]. Ses communications mettent en avant les usages institutionnels, les actifs numériques et la trésorerie [S4]. Ripple est une entreprise ; XRPL est un réseau dont les amendements dépendent de sa gouvernance. Le soutien de Ripple ne prouve pas une activation réseau.

ÉLÉMENT CONCURRENTIEL — Le 10 septembre 2026, Ripple annonce une extension de GSmart dans Ripple Treasury : prévisions, liquidité, risque, réconciliation, gouvernance et séparation entre calcul déterministe et interprétation IA [S5]. Octro ne doit donc pas revendiquer l'invention de la trésorerie agentique ni cette séparation comme exclusivité.

INTERPRÉTATION — Octro peut intéresser l'écosystème en démontrant comment des données externes deviennent un plan de crédit XLS-66 reproductible, avec preuves de contraintes et feedback développeur. Il peut aussi chevaucher des offres de Ripple. Le positionnement proposé est une couche d'optimisation et d'intégration spécialisée pour PSP, intégrable à un TMS existant. Ce n'est ni un partenariat acquis ni une garantie de victoire.

PREUVES À PRÉSENTER — Un déficit détecté avant l'échéance ; un plan moins coûteux que la référence dans un scénario documenté ; une nouvelle décision après retard ; LoanSet et LoanPay validés ; un refus sécurisé ; un dossier de reproductibilité. RLUSD peut devenir un actif cible lorsque réseau, émetteur, accès et cadre légal sont validés. XRP reste nécessaire aux frais et réserves selon les règles réseau ; aucune thèse de prix XRP n'est requise.

5 Cas d'usage mondiaux et modèle économique

Cas initial — PSP ou marketplace : sorties clients avant règlement acquéreur. Sources : calendrier de payout, solde bancaire, obligations. Action : facilité courte autorisée. Risques : chargebacks, retard, concentration, double financement. Le prêteur doit vérifier les droits sur les créances.

Europe — Trésorerie de marketplace, paiements fournisseurs et encaissements différés. Commencer avec une société et un partenaire habilité ; distinguer EUR bancaire et actif USD sur XRPL. Pas de conversion fictive à parité.

Afrique de l'Est ou de l'Ouest — Besoin de float d'un opérateur mobile money. Un connecteur M-Pesa ou Wave peut normaliser les événements mais les autorisations, liquidités et règles de change sont propres au pays. MVP : simulateur documenté, aucun partenariat revendiqué.

Amérique latine — Marchand recevant via PIX ou acquéreur et payant un fournisseur à une autre date. Le crédit répond au calendrier réel ; la rapidité du rail ne supprime pas le besoin de fonds de roulement. Inde et Asie du Sud-Est : même principe avec UPI, plateformes ou flux export, sous contraintes locales.

Extensions — Factures B2B, supply chain, fonds de roulement de marchands, trésorerie intersociétés. Les cycles agricoles exigent un horizon plus long ; les freelances et travailleurs indépendants impliquent potentiellement des personnes physiques et un cadre distinct. Ces segments sont hors MVP.

Modèle économique proposé : abonnement B2B pour supervision et simulation, puis tarification d'usage API et frais d'intégration. Une rémunération liée au crédit exige une analyse des rôles réglementaires et des conflits d'intérêts. L'optimiseur MUST afficher les frais et ne pas favoriser un vault rémunérant Octro au détriment de l'objectif convenu.

6 MVP et limites de portée

P0 obligatoire — Une organisation fictive, une devise économique de simulation, deux sources normalisées (PSP et banque simulés), trois scénarios de règlement, une policy versionnée, horizon glissant 72 h, optimisation linéaire et indicateur CVaR sur scénarios. Une interface React Native via Expo Web couvre connexion, trésorerie, proposition, approbation et suivi.

P0 XRPL — Un vault réel sur réseau de développement compatible, dépôt, broker, création de prêt et remboursement avec preuves de validation. Si le réseau choisi ne permet pas le lending, le statut est « démonstration simulée » et le critère XRPL demeure non satisfait. Une vidéo antérieure identifiée sert de secours, sans faire croire à une exécution en direct.

P0 agentique — Un orchestrateur avec deux rôles spécialisés, Analyst et Explainer, utilisant un serveur MCP interne à namespaces séparés. Le chemin manuel reste fonctionnel sans modèle. P0 sécurité : identité de test, cloisonnement tenant, refus hors policy, idempotence, signature isolée, audit et arrêt d'urgence.

P1 — Second vault dans la comparaison, expérience mobile sur appareil, DIDSet puis résolution et preuve de contrôle, scénario de credential révoqué simulé. P2 — Credentials natifs et domaines restreints lorsque supportés, second connecteur réel, financement réparti sur plusieurs prêts.

Hors MVP — Fonds réels, lancement mondial, conversion fiat automatique, crédit grand public, modèle ML entraîné, garanties probabilistes calibrées, HSM de production, multi-région, app stores, ZK, création autonome de connecteurs en production. L'architecture les anticipe sans les présenter comme livrés.

7 Parcours produit et critères UX

Parcours trésorier — Choisir l'organisation et le contexte réseau ; consulter sources et fraîcheur ; voir la courbe de solde avant/après plan et la réserve ; ouvrir le manque ; comparer montant, coût total, horizon, scénarios et contraintes ; demander validation ; signer ; suivre confirmation et remboursement. L'écran distingue « prévu », « approuvé », « soumis » et « confirmé ».

Parcours prêteur — Consulter capital disponible, exposition active et réservée, limites, dossier économique et échéancier ; approuver ou refuser les termes ; suivre remboursements et incidents. Le MVP limite ce parcours à une vue interne de broker, pas à un marché public de crédit.

Parcours incident — Une entrée attendue est retardée. Le système recalcule, affiche le nouveau déficit et explique la règle bloquante. Il ne modifie pas le prêt existant. Toute nouvelle facilité nécessite un nouveau dossier et les autorisations requises.

Parcours identité — Vérification de test du représentant ; rattachement organisation ; contrôle de wallet par challenge signé ; éventuelle attestation ; date d'expiration visible. « Identité vérifiée » ne signifie pas « crédit accordé ».

Recette UX : l'utilisateur identifie en moins de 30 secondes le montant manquant, le moment du manque et la prochaine action dans un test modéré. Chaque écran possède états vide, chargement, erreur, périmé, interdit et reprise. Le refus montre un code et une explication pratique. Les preuves XRPL sont accessibles dans un panneau détaillé ; aucune donnée de test n'est présentée comme solde réel.

8 Design humain et maquettage Pencil

Direction proposée à partir de l'intention American Matcha : fond crème #F5F2E8, texte encre #20271F, vert profond #304B35, accent matcha #B8C98A, erreur terre #9F3F31. Ce vocabulaire traduit une ambiance chaleureuse et éditoriale ; la référence visuelle exacte American Matcha reste à confirmer avant fidélité graphique. Ne pas copier une marque ou supposer qu'une recherche homonyme constitue la référence.

Typographie : une sans-serif lisible pour montants, formulaires et tableaux ; une serif discrète facultative pour les titres éditoriaux. Chiffres tabulaires, corps minimum 16 px sur mobile, grille de 8 px, espacements 8/16/24/32, coins 8 à 12 px. Les couleurs seules ne portent jamais un état. Vérifier contraste WCAG AA, clavier, focus, VoiceOver/TalkBack et réduction des animations.

Interdits produit : badges « AI powered », robots, étoiles magiques, gradients violet-cyan génériques, faux indicateurs de réflexion et slogans vagues. Écrire « Il manque 90 000 demain » et « Voir les hypothèses ». Ne pas remplacer les informations financières par une conversation obligatoire.

Pencil.dev, redirigé vers pen.dev lors de la vérification [S15], sert au maquettage. Kevin livre les fichiers de conception, composants, tokens JSON et captures de cinq écrans aux largeurs 390 et 1440. Les états critiques et le flux de signature sont maquetés. La génération de code éventuelle est un point de départ à tester ; aucun export React Native parfait n'est supposé.

9 Architecture et stack décidée

Architecture MVP : monorepo modulaire, API centrale et workers, avec un service Python de calcul isolé. Flux : React Native → API → données et policy → moteur Python → plan immuable → approbation → adaptateur XRPL → signataire → ledger → réconciliation. Les agents appellent les mêmes services via MCP ; ils ne constituent pas un second chemin d'autorisation.

Frontend : React Native, Expo, Expo Router et TypeScript ; priorité Expo Web responsive, builds Android/iOS après stabilisation. TanStack Query pour état serveur, React Hook Form et validation de schémas pour formulaires, stockage sécurisé natif des jetons ; sur web préférer session cookie HttpOnly via API même origine. Les graphiques doivent fonctionner sur web et natif, avec résumé textuel accessible [S16].

Backend : Node.js LTS supporté à l'installation, TypeScript strict, Fastify, OpenAPI 3.1, JSON Schema, accès PostgreSQL via migrations SQL et couche de requêtes typée. Workers PostgreSQL avec outbox et verrouillage SKIP LOCKED pour éviter d'ajouter Redis au MVP. Python avec FastAPI, Pydantic, NumPy et SciPy/HiGHS pour le programme linéaire. Les versions exactes seront figées dans lockfiles et images après smoke test sur la VM ; ne jamais dépendre de latest en CI.

Agentique : graphe d'états explicite en TypeScript, appels de modèle derrière ModelGateway interchangeable et SDK MCP officiel validé par tests de contrat. Pas de dépendance obligatoire à un fournisseur de LLM. Infra : Docker Compose, reverse proxy Caddy ou équivalent existant, TLS, volumes persistants, observabilité légère. Aucun Kubernetes pour trois personnes au hackathon.

10 Utilisation de la VM existante

Inventaire déclaré : VM, PostgreSQL, MongoDB, stockage S3, domaine, serveur SMTP et Stripe Identity. Samet vérifie versions, OS, vCPU/RAM, espace, réseau, TLS, droits, sauvegardes, région et accès aux secrets avant installation. Cible de départ proposée : 4 vCPU, 8 à 16 Go RAM et disque SSD ; à mesurer, pas une description de la VM actuelle.

PostgreSQL est obligatoire pour comptes, événements normalisés, état financier, jobs, politiques et audit. MongoDB MAY conserver des documents de connecteurs variables et des brouillons agentiques expurgés, avec TTL ; il ne décide jamais d'un solde, d'une permission ni de l'état d'un prêt. Si S3 couvre les données brutes, désactiver MongoDB dans le chemin MVP réduit la charge.

S3 reçoit snapshots chiffrés, exports et pièces strictement nécessaires ; buckets privés, liens présignés courts, versionnement et cycle de vie. Un stockage compatible S3 sur la même VM n'est pas une sauvegarde indépendante : prévoir copie hors hôte. SMTP sert aux invitations et alertes minimales avec SPF/DKIM/DMARC ; les notifications ne déclenchent aucune autorisation.

Domaine : app et API même origine si possible, sous-domaines séparés pour monitoring privé et callback identity. Bases jamais exposées publiquement ; ingress public limité au HTTPS ; administration par VPN ou SSH avec clés et filtrage. Conteneurs non root, réseau interne pour Python, MCP et bases. Ressources CPU/mémoire plafonnées pour éviter qu'un solveur bloque l'API.

Stripe Identity utilise son mode test au hackathon. La disponibilité annoncée ne prouve pas que clés, webhooks ou contrats soient configurés. Ne pas stocker de clés dans le dépôt ni dans les maquettes.

11 Modèle de données et invariants

Entités relationnelles : Organization, User, Membership, Consent, DataConnection, FinancialAccount, EconomicEvent, CashSnapshot, Receivable, ExposureReservation, ForecastRun, Scenario, PolicyVersion, OptimizationRun, Plan, Approval, Execution, VaultReference, LoanReference, Repayment, IdentityCheck, DID-Binding, CredentialStatus, AuditEvent, OutboxEvent.

Chaque objet métier porte `tenant_id`, `id` UUID, `created_at` UTC, `schema_version` et provenance utile. Les relations inter-tenant sont interdites par contraintes et contrôles serveur. Les événements ont `source_event_id` unique par tenant et connexion, `occurred_at`, `observed_at`, `expected_settlement_at`, `status` et `raw_object_ref`. Un règlement réconcilie sa créance au lieu de créer deux revenus.

Montants : chaînes décimales dans JSON et NUMERIC(38,18) dans PostgreSQL ; jamais de float binaire pour la comptabilité. `AssetId` distingue fiat, XRP et token émis : réseau, code et émetteur, ou identifiant MPT selon actif. La conversion en drops ou précision protocolaire appartient à l'adaptateur. Quantifier et révérifier toutes les contraintes après arrondi du solveur ; un reliquat ne peut dépasser une limite.

Journal financier en partie double par actif, avec écritures immuables et écritures de correction. La somme débit/crédit doit être nulle pour chaque transaction comptable. Une prévision est stockée séparément des soldes confirmés. Les sommes en transit et engagements en attente consomment la capacité ; les fonds clients cantonnés ne financent pas la trésorerie propre.

Snapshot de décision : références d'événements, hash SHA-256, calendrier, modèle, scénarios, seed, policy, taux et frais, version du solveur et code commit. Les dossiers sont immuables ; une modification produit une nouvelle version et invalide les approbations précédentes.

12 Prévisions et jumeau de trésorerie

Le jumeau est un état opérationnel réconcilié : cash disponible par compte/actif, obligations ordonnées, créances attendues, dette et réserve. Il ne nécessite ni simulation 3D ni modèle génératif. Le premier forecast utilise échéanciers connus et scénarios de retard paramétrés ; le LLM n'invente pas une probabilité d'encaissement.

Scénarios MVP : règlement normal à H+36 (poids 0,80), tardif à H+60 (0,15), absent sur l'horizon (0,05). Ces poids sont fictifs et affichés comme tels. Leur somme vaut 1. Les retards corrélés de plusieurs créances doivent être représentés ensemble ; l'indépendance ne doit pas être supposée par défaut.

Fraîcheur proposée : solde et liquidité exécutable âgés de moins de 60 secondes au moment de préparer ; événements PSP de moins de 15 minutes pour une nouvelle proposition ; policy et conformité revérifiées à chaque action financière. Les seuils restent configurables par source. Sans source fiable, mode consultation/simulation et blocage du financement.

Post-hackathon : estimer les distributions de délais et défauts avec historique autorisé, séparation temporelle entraînement/validation, suivi des quantiles et tests de couverture. Mesurer erreur absolue des montants, erreur des dates et calibration des probabilités. Comparer à une baseline calendrier avant de retenir un ML plus complexe. Une cible de liquidité à 99 % n'est pas une promesse de solvabilité à 99 %.

Toute nouvelle arrivée de fonds réels déclenche réconciliation, recalcul et proposition de service du prêt. Une anomalie ou une dérive n'autorise pas une modification unilatérale du contrat.

13 Formulation MPC et optimisation sous contraintes

Le Model Predictive Control recalcule un plan sur un horizon mobile et ne met en œuvre que la première action autorisée [S11]. À chaque cycle : figer l'état, produire les scénarios, résoudre, vérifier les contraintes en précision décimale, demander validation si nécessaire, puis observer. Il s'agit ici d'une application proposée au financement, sans prétendre aux garanties de stabilité d'un procédé physique.

Notation : t désigne l'heure, s un scénario de probabilité p_s , a un actif, v un prêteur. C est le cash, I les entrées, O les sorties dues, b les tirages, r le service contractuel de dette et B la réserve. Les flux de conversion incluent taux, frais et délais.

$$C[t+1,s,a] = C[t,s,a] + I[t,s,a] - O[t,a] \\ + \sum_v b[t,v,a] - \sum_v r[t,s,v,a] + FXnet[t,s,a].$$

Contraintes MUST : principal et tirage positifs, capacité disponible du vault, plafond de crédit approuvé, exposition engagée plus active, restrictions d'actif et de juridiction, paiements non reportables, durée autorisée, service de dette et réserve. Une variable de déficit q mesure $\max(0, B-C)$ pour diagnostic et risque ; elle ne rend jamais un paiement impossible « autorisé ».

$$\min E[\text{coût crédit} + \text{coût change} + \text{frais}] + \lambda \times CVaR\alpha(L).$$

Les coûts et la perte L sont exprimés dans une devise numéraire au taux versionné. L est le déficit maximal sous réserve sur l'horizon, après conversion documentée. λ est sans unité. Le coût inclut intérêts, origination, clôture anticipée, frais réseau et coûts de conversion pertinents. La valeur terminale des dettes restant après $H+72$ est intégrée ; on ne cache pas une échéance en sortie d'horizon.

Le MVP n'autorise pas de décisions futures dépendant d'informations encore inconnues : plan commun aux scénarios pour les décisions engagées. Une future version stochastique avec recours doit imposer la non-anticipativité avant la révélation des événements. En cas d'infaisabilité : retourner contraintes bloquantes et état INFEASIBLE, aucun ordre financier.

14 Min cost flow CVaR et exemple calculé

Le min-cost flow représente la liquidité par un graphe étendu dans le temps : nœuds compte/actif/heure ; arcs de détention, financement et transfert avec capacité, délai et coût. Il convient aux coûts linéaires dans une unité cohérente. Des changes à taux différents, frais fixes, échéances de prêt et contraintes CVaR nécessitent un LP généralisé ou un MILP ; ce n'est plus un simple min-cost flow. Le MVP utilise un LP, sans frais fixes conditionnels ni allocation multi-actif réelle.

CVaR au niveau alpha proposé de 0,95 : introduire η et z_s . Minimiser $\eta + \sum_s p_s z_s / (1 - \alpha)$, avec $z_s \geq L_s - \eta$ et $z_s \geq 0$ [S12]. Le moteur peut imposer $CVaR \leq \text{risk_budget}$ et minimiser le coût parmi les plans admis. Un budget de risque nul peut rendre le scénario d'absence de paiement infaisable ; le produit doit le montrer.

Exemple pédagogique sans frais ni change : besoin 90 000 pendant 36 h, offre A limitée à 60 000 à 4,2 % annuel simple, offre B à 4,8 %. Base ACT/365 : A coûte $60000 * 0,042 * 36 / 8760 = 10,36$; B sur 30 000 coûte 5,92 ; total 16,27 après calcul puis arrondi global. B seule coûte 17,75. Gain illustratif 1,48. Cet exemple n'est pas le calcul contractuel XLS-66, qui doit venir des règles du protocole et inclure ses frais.

Après tirage et paiement, la trésorerie vaut 10 000 ; après encaissement de 110 000 et remboursement de 90 016,27, elle vaut 29 983,73. Si le règlement n'arrive pas, le remboursement n'est pas financé : le risque reste réel. Les trois scénarios de démonstration ne suffisent pas à calibrer une queue de distribution en production.

15 Agents IA et outils MCP

Le Coordinator gère un graphe borné : collecter → analyser → simuler → expliquer → attendre décision → suivre. Analyst appelle snapshot, forecast et optimize ; Explainer transforme les résultats structurés en texte avec références aux champs. Ces rôles partagent un service de calcul, pas un raisonnement financier libre. Maximum proposé : 12 appels outils, 30 secondes par exécution interactive et budget de tokens configuré. Au dépassement, retour au dashboard avec état sauvegardé.

Namespaces MCP : data.get_snapshot, data.sync_source, engine.forecast, engine.optimize, policy.evaluate, identity.get_eligibility, xrpl.get_vault, xrpl.get_loan, xrpl.prepare_loan, monitor.get_incidents. Les outils de lecture sont accessibles selon scopes ; sync et prepare sont des écritures contrôlées. Le signataire et submit ne sont pas exposés au LLM. Chaque outil réutilise l'autorisation métier de l'API.

Entrée standard : request_id, références d'objets et expected_version ; tenant et identité effective dérivés du jeton. Sortie : status, data, evidence_refs, warnings et trace_id. Pas de SQL arbitraire, shell, URL libre ni accès aux secrets. Le serveur valide JSON Schema, limites de taille, quota et egress.

Le protocole MCP normalise les appels ; il n'est ni un moteur d'agents ni une preuve de confiance. Pour un serveur distant : HTTPS, authentification adaptée, jetons courts et audience vérifiée, scopes minimaux, pas de transmission aveugle de tokens [S13]. En MVP, services privés et identités de service distinctes suffisent au déploiement interne documenté.

Instructions contenues dans factures, transactions ou réponses d'outils = données non fiables. Les résumés doivent référencer un plan existant ; si chiffres générés et chiffres structurés divergent, afficher le rendu déterministe. Les journaux conservent outils, décisions et preuves utiles, pas de chaîne de pensée privée.

16 Intégration XRPL et disponibilité réelle

XLS-65 définit le vault mono-actif ; XLS-66 décrit des prêts à terme avec underwriting hors chaîne [S2, S3]. L'adaptateur sélectionne une version compatible de xrpl.js et du schéma ledger. La documentation consultée affiche des statuts d'amendement chargés dynamiquement et des réserves sur l'approbation [S1, S6] : aucune activation Mainnet n'est certifiée par ce CDC.

Gate G0 : Augustin consigne endpoint, network_id, version serveur, ledger validé, amendements actifs et versions SDK. Puis il exécute un smoke test de vault et lending sur le réseau exact du hackathon. Un statut inconnu bloque le mode réel ; un réseau dev n'implique pas que tous les amendements soient actifs. Les identifiants de réseau sont obligatoires dans toutes les références.

Chemin de démonstration : comptes de test financés → VaultCreate → VaultDeposit → LoanBrokerSet et couverture requise → termes et signatures LoanSet → validation → lecture de l'objet Loan → LoanPay → validation et nouveau solde. Les signatures du borrower et du broker suivent le schéma exact de LoanSet ; ne pas substituer une simple approbation UI à une signature protocolaire.

Les mappings principal, taux, période, nombre de paiements, grâce et frais sont encapsulés et testés dans l'adaptateur. Pas de payload de production issu d'un ancien exemple v1.1 non révérifié. Les calculs de remboursement utilisent l'échéancier ledger et les paramètres contractuels ; LoanPay impose notamment l'identité du borrower et des règles de montant/timing [S10].

Une réponse submit positive ne vaut pas règlement : exiger transaction validée, résultat tesSUCCESS et état attendu. Garder frais/réserves XRP suffisants. Pour un token émis, vérifier émetteur, trustline, freeze, clawback et autorisations pertinentes. Un token de démo n'est jamais étiqueté RLUSD réel.

17 DID et identité vérifiable

Décision : OUI à une intégration DID facultative P1, dédiée d’abord aux organisations et comptes de test. XRPL documente les DID et les transactions DIDSet/DIDDelete [S6, S7]. Un DID identifie un sujet contrôlant des clés ; il ne prouve pas à lui seul son identité légale, sa solvabilité ou son habilitation à emprunter. Les principes W3C distinguent contrôle, méthodes de vérification et attestations [S9].

Architecture proposée : DIDBinding relie organisation, compte XRPL, réseau, DID, preuve de contrôle, date de vérification et statut. Le document DID public se limite aux clés et points de service nécessaires ; il peut être référencé par URI selon l’implémentation. Aucun nom, email, document d’identité, identifiant Stripe ou empreinte de pièce sensible sur le ledger. Une empreinte peut elle-même être corrélable.

Enrôlement : vérification du représentant hors chaîne ; validation KYB et mandat séparés ; challenge unique lié au domaine, à la session, au tenant et au réseau, expiration 5 minutes ; vérification de signature ; création ou résolution du DID ; attestation d’éligibilité par un émetteur autorisé. La syntaxe did:xrpl et le resolver sont testés contre la méthode et le réseau effectifs ; ne pas fabriquer une URI à partir d’une adresse sans vérifier la méthode.

DID, VC W3C et Credentials XRPL sont distincts. Les Credentials natifs utilisent leurs propres structures [S8] ; ils peuvent servir de contrôle d’accès avec des fonctionnalités compatibles. Aucun accès à un vault n’est réputé protégé par un DID seul. Prévoir un registre d’émetteurs acceptés, durée, statut de révocation et audience d’utilisation.

Tests P1 : création/résolution, mauvais signataire, replay de challenge, expiration, mauvais réseau, rotation des clés, indisponibilité du document et révocation d’attestation. Rotation ou compromission suspend les nouvelles actions, déclenche revalidation et conserve l’historique. Supprimer le DID ne supprime pas l’historique public. Un mode sans DID reste pleinement utilisable avec identité et autorisations classiques.

18 Stripe Identity conformité et données personnelles

Stripe Identity fournit des vérifications d'identité selon les fonctions et pays supportés [S14]. Ce composant ne constitue pas à lui seul un KYB complet, un contrôle des bénéficiaires effectifs, un screening de sanctions, une licence ni une décision de crédit. Stocker seulement session_id, statut, horodatage et références minimales ; privilégier la collecte hébergée. Vérifier la signature du webhook et son idempotence ; ne jamais faire confiance à la redirection navigateur.

Gouvernance proposée : Octro opère la technologie ; le partenaire prêteur décide, contractualise et porte le rôle de crédit défini juridiquement. La réalité des services, du contrôle des clés et de la rémunération prime sur cette formulation. Un conseil local doit déterminer licences de prêt, intermédiation, services de paiement, custody et obligations crypto applicables avant tout fonds réel.

Union européenne : documenter RGPD, finalités, base légale par traitement, minimisation, droits, rétention, sous-traitants, transferts et éventuelle AIPD [S17]. Le consentement de connexion à une source n'est pas une base légale universelle. Analyser MiCA pour les services et actifs concernés, sans supposer qu'il couvre toutes les règles de crédit [S18]. L'AI Act mentionne la solvabilité des personnes physiques parmi les usages à haut risque ; l'élargissement à des indépendants exige une qualification spécifique et une revue du calendrier applicable [S19].

Gates de lancement : pays emprunteur/prêteur, entité contractante, actif, corridor, KYB/UBO, sanctions/PEP selon obligations, licence, disclosures, plafonds/tarifs, recouvrement, conservation et localisation. À défaut d'une décision documentée : pas de prêt réel. Une infrastructure mondiale nécessite une matrice de pays, pas un badge « compliant globally ».

MVP : données synthétiques uniquement. Politique proposée hors production : traces opérationnelles 30 jours, raw de test 7 jours, snapshots de recette conservés avec la version. La rétention de production est définie avec les obligations légales, y compris les exceptions à l'effacement. Aucun dossier KYC envoyé au LLM.

19 API et contrats machine readable

API REST /v1 ; authentification OIDC/session, RBAC et contrôle tenant serveur. Les GET retournent ressource, version et fraîcheur. Les commandes longues retournent 202 avec `operation_id`, puis GET /v1/operations/{id}. Les erreurs exposent code stable, message, retryable, details et `trace_id`, sans secret. OpenAPI décrit tous les schémas et exemples ; JSON Schema interdit les champs inattendus sur les commandes critiques.

Endpoints MVP : POST /v1/connections/sync ; GET /v1/cash-snapshots/{id} ; POST /v1/forecasts ; POST /v1/plans ; GET /v1/plans/{id} ; POST /v1/plans/{id}/approvals ; POST /v1/executions ; GET /v1/executions/{id} ; POST /v1/loans/{id}/repayment-plans ; GET /v1/audit-events. Identité P1 : POST /v1/did-challenges puis /v1/did-bindings. Webhook Stripe dédié avec authentification par signature fournisseur.

Toute commande monétaire MUST exiger Idempotency-Key, `expected_version` et `plan_hash` applicable. Clé unique par tenant, route et commande métier ; conserver statut et réponse avec empreinte du corps. Même clé/autre corps retourne 409. Les montants utilisent `amount_decimal` et `asset_id`, jamais une chaîne ambiguë « \$ ».

Codes minimaux : DATA_STALE, POLICY_DENIED, COMPLIANCE_REQUIRED, INFEASIBLE, PLAN_EXPIRED, VERSION_CONFLICT, INSUFFICIENT_LIQUIDITY, NETWORK_UNSUPPORTED, SIGNATURE_REJECTED et LEDGER_OUTCOME_UNKNOWN. Aucun retry aveugle de ces deux derniers cas.

Le paquet joint contient un manifeste d'exigences, un schéma de plan et un exemple. Ces fichiers décrivent les contrats Octro, pas les payloads natifs XRPL. Le contrat OpenAPI complet doit être implémenté en premier et vérifié par Samet/Augustin avant les intégrations.

20 États approbations et signatures

Cycle plan : DRAFT → DATA_READY → OPTIMIZED → POLICY_PASSED → COMPLIANCE_PASSED → APPROVAL_PENDING → APPROVED → PREPARED → SIGNATURE_PENDING → SIGNED → SUBMITTED → CONFIRMED. Branches : REJECTED, EXPIRED, CANCELLED, FAILED et OUTCOME_UNKNOWN. Seul CONFIRMED déclenche l'état de prêt ACTIVE ; servicing suit ACTIVE → DUE → REPAYED ou LATE → IMPAIRED/DEFAULTED selon événements et droits protocole.

L'approbation porte le hash des termes, frais maximaux, réseau, compte, destinataires, policy, version du plan et expiration proposée 5 minutes. Toute modification exige une nouvelle approbation. Relire liquidité, exposition et conformité avant préparation et soumission ; une réserve interne ne garantit pas que le vault externe soit encore liquide.

Lecture/simulation : Analyst autorisé. Proposition : trésorier. Approbation de termes : approbateur désigné et prêteur selon responsabilités. Signature : comptes requis par XRPL. Changement de policy et clés : administrateur habilité avec seconde revue. Production : séparation proposant/approbateur obligatoire ; démo permet les mêmes personnes dans des comptes de rôle clairement distincts.

Le service de signature isolé n'accepte qu'un payload de type autorisé conforme au plan et à un jeton d'approbation à usage unique. Aucune seed dans LLM, MCP, frontend ou logs. Clés de test jetables avec accès restreint ; production via wallet utilisateur ou service institutionnel/HSM et procédures de récupération validées.

En cas de timeout après envoi : persister blob signé, hash et bornes de validité avant submit, passer OUTCOME_UNKNOWN, rechercher la transaction et réconcilier. Réenvoyer le même blob peut être permis par la stratégie protocole ; générer une nouvelle transaction attend la preuve que l'ancienne ne peut plus être validée. Les séquences sont sérialisées par compte ; les tickets éventuels sont gérés explicitement.

21 Sécurité et menaces

Menaces principales : prise de compte, accès inter-tenant, faux webhook, injection dans document, outil MCP compromis, vol de clé, double tirage, donnée périmée, opérateur malveillant et exposition de pièces. Les barrières doivent être appliquées par des services déterministes indépendants du texte produit par le modèle.

Identité applicative : MFA pour rôles sensibles, sessions courtes, révocation, authentification renforcée avant signature, protection CSRF si cookies et CORS limité. Autorisation à chaque objet, row-level security ou contrôles équivalents testés, rôle DB distinct pour worker et API. Les accès support sont temporaires et audités.

Entrées : requêtes préparées, validation de schéma, limites de taille, scan des fichiers et quarantaine avant traitement. Protection SSRF sur connecteurs, resolvers DID et liens de documents : destinations autorisées, blocage réseaux privés et métadonnées cloud, contrôle des redirections. Désactiver l'exécution de contenu et les dépendances distantes non nécessaires.

Secrets : gestionnaire ou fichiers montés à permissions strictes au MVP ; séparation environnements, rotation, chiffrement au repos avec clés séparées des backups, TLS en transit. Les logs masquent jetons, seeds, données KYC et valeurs sensibles. Signature des images et inventaire des dépendances en CI ; aucune installation dynamique d'outil par l'agent.

Fraude crédit : identifiant unique des créances, réservations transactionnelles et contrôle de l'exposition empêchent le double engagement interne. Ils ne prouvent pas l'absence de financement chez un tiers : vérifications externes et déclarations contractuelles restent nécessaires. Aucune donnée protégée ou sans pertinence économique n'entre dans le moteur. Les explications exposent les facteurs contestables et un parcours de revue humaine.

22 Résilience observabilité et exploitation

Fail-safe : données périmées, policy absente, identité incertaine, solveur invalide ou réseau non supporté bloquent les nouvelles actions financières. Une panne du modèle n'interrompt pas les vues, simulations déterministes et échéances déjà suivies. Un kill switch bloque préparation et soumission de nouvelles actions ; il ne révoque pas une transaction déjà acceptée par le réseau.

Jobs et outbox sont écrits dans la même transaction que le changement métier. Livraison au moins une fois, handlers idempotents, backoff exponentiel avec jitter et file d'échec ; jamais promesse d'exactly-once distribuée. Les verrouillages d'exposition sont transactionnels ; perte de lock ou conflit de version annule la préparation.

Pannes : timeout solveur après 5 secondes retourne TIMEOUT et aucune exécution ; solde insuffisant renvoie une nouvelle proposition ; provider dégradé sert les données avec âge ; SMTP indisponible conserve l'alerte dashboard ; crash worker reprend depuis état durable ; désaccord RPC conserve OUTCOME_UNKNOWN. Le remboursement en retard appelle intervention, pas emprunt caché pour refinancer.

OpenTelemetry relie API, job, solveur, outil et transaction par trace_id. Logs structurés, métriques Prometheus et tableau Grafana privé : âge des sources, latence p95, solutions infaisables, écarts prévus/réalisés, échecs de policy, coût LLM, âge du dernier ledger, états inconnus et retard de jobs. Alertes : résultat inconnu > 60 s, source expirée, zéro ledger récent, backup en échec, disque > 80 %.

Cibles MVP à tester : p95 lecture < 500 ms hors réseau, optimisation < 5 s pour 72 pas/3 scénarios/2 sources, 10 sessions simultanées. Pilote : disponibilité API 99,5 % mensuelle, RPO 15 min et RTO 4 h, à valider par restauration. La VM unique reste un point de panne ; ces objectifs ne sont pas des SLA acquis. Backup PostgreSQL et S3 hors hôte, restauration isolée, puis réconciliation ledger avant réouverture.

23 CI CD et organisation du dépôt

Monorepo : apps/client ; apps/api ; apps/worker ; services/optimizer ; packages/contracts ; packages/domain ; packages/xrpl ; packages/agents ; packages/mcp ; packages/ui ; infra ; fixtures ; docs ; tests. Les modules domain et optimizer ne dépendent pas du LLM. Les contrats appartiennent à Samet avec revue Augustin ; Kevin consomme des exemples stables pour ne pas attendre les services.

Pipeline PR : lint et types, validation JSON Schema/OpenAPI, tests unitaires financiers, tests de policy et états, tests PostgreSQL/outbox, scan secrets et dépendances, build client/API/optimizer, génération SBOM. Tests réseau dédiés séparés des tests déterministes ; un réseau indisponible est marqué indisponible, pas faussement réussi.

Déploiement : images référencées par digest ; CI sans secret de production dans les PR ; environnement de staging ; migrations expand/contract et snapshot avant opération risquée ; smoke tests de lecture, policy et network capability ; promotion de la même image. Les secrets de déploiement ont des droits limités. Production nécessite le gate de lancement défini au chapitre conformité.

Rollback : revenir à l'image compatible précédente et désactiver nouvelles exécutions. Ne jamais « rollback » un prêt ledger par suppression SQL ; réconcilier et, si nécessaire, réaliser une opération compensatoire autorisée. Les migrations destructrices nécessitent une stratégie explicite hors fenêtre hackathon.

Source de vérité pour Claude/Codex : ce CDC et requirements.json ; contrats d'API pour les formats ; schémas XRPL épinglés pour les transactions réseau. Tout conflit crée une décision ADR explicite plutôt qu'une invention. Chaque tâche de développement cite un ID d'exigence et son test d'acceptation ; aucun agent n'ajoute un flux d'argent, pays ou permission hors périmètre pour satisfaire une démo.

24 Plan de tests et Definition of Done

Financier : besoin nul → aucun tirage ; exemple 90 000 → réserve conservée ; plafond 80 000 → infaisable ; coût A/B et frais → choix conforme ; retard → recalcul ; données absentes → blocage. Tester arrondis, actifs 0/2/6/18 décimales, dates UTC/changement d'heure, échéance à H+72 et dette terminale. Les frais fixes non modélisés provoquent refus de l'offre, pas optimisation erronée.

Risque : poids invalides refusés, CVaR comparée à calcul manuel, scénario extrême déclenche alerte, décisions non anticipatives, explication identique aux valeurs du plan. La CVaR de pertes $[0,0,100]$ aux poids $[0,80;0,15;0,05]$ et alpha 0,95 vaut 100. Les contraintes post-arrondi sont toutes revérifiées.

Sécurité : un tenant ne lit pas un autre plan ; Analyst ne signe pas ; faux webhook rejeté ; challenge DID rejoué rejeté ; credential expiré bloque l'éligibilité ; injection « ignore policy » reste du texte. Chaque refus possède un code d'erreur et une trace.

Résilience : deux POST identiques → une seule exécution ; même clé/autre corps → 409 ; crash après submit → recherche par hash ; approbation périmée → blocage ; vault vidé entre plan et signature → nouvelle proposition ; panne LLM → parcours manuel ; reset ne touche que données et comptes de test dédiés.

Recette XRPL : preuve network_id, hashes, ledger index, résultat final et état avant/après pour dépôt, prêt et remboursement. Recette finale : trois replays synthétiques reproductibles, tous tests P0 verts, cinq écrans utilisables, aucun secret détecté, README d'installation, vidéo de secours identifiée, registre des limites et feedback XRPL. Un critère non satisfait reste visible ; le nombre de primitives ne remplace pas la valeur démontrée.

25 Répartition Kevin Samet Augustin

Kevin — Responsable produit, définition du scénario, maquettes Pencil, tokens, React Native et expérience d’approbation. Implémente Coordinator/Analyst/Explainer et branche MCP sur contrats fournis. Livre cinq écrans, explications fondées sur le plan, parcours manuel sans LLM et pitch. Reçoit de Samet les fixtures API et d’Augustin les sorties du solveur ; ne recode pas les calculs dans l’UI.

Samet — Responsable VM, réseau, PostgreSQL, S3, usage optionnel MongoDB, authentification, API, jobs/outbox, identité Stripe, audit, CI/CD et observabilité. Livre migrations, contrôle tenant, idempotence, registry d’approbations, environnement reproductible et restauration testée. Donne les contrats au début du hackathon. Revoit avec Augustin le stockage des transactions et avec Kevin les erreurs UX.

Augustin — Responsable prévision déterministe, LP/MPC/CVaR, tests numériques, adaptateur XRPL, network gate, vault/broker/prêts, remboursement et réconciliation ledger. Livre exemples chiffrés, schéma des plans et preuves réseau. Implémente DID P1 seulement après chemin de prêt fiable. Ne porte pas seul le backend ni l’UI.

Interfaces communes à figer à H+2 : EconomicEvent, CashSnapshot, Plan, Approval, Execution et Error. Samet fournit un serveur mock ; Augustin fournit trois fixtures de scénarios ; Kevin développe dessus. Revues croisées obligatoires : signature et exposition par Samet+Augustin ; chiffres affichés par Kevin+Augustin ; contrôle d’accès par Samet+Kevin.

Arbitrage : Kevin priorise le produit ; Augustin peut bloquer un résultat financier invalide ; Samet peut bloquer une faille de données ou d’autorisation. Un désaccord sur argent ou sécurité maintient le blocage et crée une décision écrite. Réduire d’abord DID, multi-vault et finitions ; conserver données → calcul → approbation → LoanSet → LoanPay.

26 Roadmap hackathon et après

H0-H4 — Tous : contrats et scénario ; Samet prépare environnement et fixtures ; Augustin exécute gate réseau et première transaction ; Kevin fige maquettes et navigation. Gate : capacité lending démontrée ou risque explicite avec plan de secours. Ne pas attendre la fin pour découvrir l'incompatibilité réseau.

H4-H12 — Samet données/API/outbox ; Augustin prêt et remboursement minimaux ; Kevin écrans sur mocks. H12-H24 — moteur et policy, connexion réelle des écrans, états d'exécution. Gate H24 : parcours manuel bout en bout et calcul reproductible.

H24-H34 — orchestration/MCP, retard de règlement, refus et réconciliation ; Samet sécurise autorisations et monitoring. H34-H40 — tests d'échec et, seulement si P0 vert, DID. H40-H48 — gel fonctionnel, répétition, captures, vidéo et feedback. Réserver au moins 20 % du temps à l'intégration et à la recette.

Semaines 1-2 — Nettoyer dette technique, fixer versions, tests de contrats et mesurer performances ; réaliser entretiens PSP/trésoriers et valider la demande. Semaines 3-6 — Connecteur réel en lecture seule, historique autorisé, backtest temporel et comparaison baseline. Semaines 7-12 — Pilote sur un pays avec partenaire, validation juridique, sécurité des clés et contrat ; commencer en mode recommandation sans exécution réelle autonome.

Après pilote — Ajouter devises et prêteurs avec coûts/délais complets, modèles calibrés, résilience hors VM unique, SDK, Credentials et identité portable. L'automatisation sous mandat exige plafonds, révocation, audit et validations spécifiques. Aucun calendrier ne remplace les gates de qualité et de conformité.

27 Démonstration indicateurs et décisions ouvertes

Démo en cinq minutes : montrer obligation et encaissement décalé ; afficher besoin et coût ; comparer au financement de référence ; accepter les termes et montrer la preuve XRPL ; simuler retard et règle de blocage ; afficher remboursement validé sur scénario compatible. La durée contractuelle réelle du prêt de test peut différer des 36 heures économiques : l'indiquer sans accélérer fictivement l'horloge du ledger.

Mesures : déficit maximal et durée sous réserve ; coût réalisé incluant frais ; écart prévision/réalité ; taux d'échéances respectées ; actions manuelles par dossier ; latence ; nombre d'états inconnus résolus ; qualité de calibration après collecte réelle. Le volume de crédit seul n'est pas un objectif suffisant. Les économies sont comparées sur mêmes entrées, contraintes et période, avec méthode et limites publiées.

Décisions ouvertes et responsables : calendrier/règlement hackathon (Kevin, H0) ; référence visuelle American Matcha exacte (Kevin, avant design final) ; capacité et région VM (Samet, H0) ; disponibilité effective des services et clés test (Samet, H0) ; réseau/amendements/SDK/actif (Augustin, H4) ; paramètres de prêt et couverture broker (Augustin, H4) ; pays, prêteur, mandat et rôles légaux (équipe, avant pilote).

Livrables de dépôt attendus : README, architecture, OpenAPI, schémas, fixtures, politiques, tests, plan de reprise, SECURITY.md, ADR, guide de démo et XRPL-LENDING-FEEDBACK.md. Le feedback précise versions, étapes, résultat attendu/obtenu et proposition, sans secrets. Ce CDC définit ce qui doit être construit ; il ne certifie pas que le logiciel existe déjà.

28 Sources vérifiées et registre documentaire

Sources consultées le 12 septembre 2026. Les références officielles étayent les fonctions citées ; les choix de stack, paramètres, UX, roadmap et attributions sont des décisions proposées pour Octro. Les statuts dynamiques du réseau doivent être constatés lors de G0.

[S1] Ripple, The XRPL Lending Protocol Bringing Credit Infrastructure Onchain, 29 juin 2026. <https://ripple.com/insights/the-xrpl-lending-protocol-bringing-credit-infrastructure-onchain/>

[S2] XRPL, Single Asset Vaults. <https://xrpl.org/docs/concepts/tokens/single-asset-vaults>

[S3] XRPL, Lending Protocol. <https://xrpl.org/docs/concepts/tokens/lending-protocol>

[S4] Ripple, Institutional DeFi on XRPL, 5 février 2026. <https://ripple.com/insights/institutional-defi-on-xrpl-scaling-real-world-finance-with-xrp-at-the-core/>

[S5] Ripple, GSmart et gouvernance de trésorerie, 10 septembre 2026. <https://ripple.com/ripple-press/ripple-treasury-brings-industry-s-first-governed-ai-for-enterprise-treasury/>

[S6] XRPL, Known Amendments. <https://xrpl.org/resources/known-amendments>

[S7] XRPL, Decentralized Identifiers. <https://xrpl.org/docs/concepts/decentralized-storage/decentralized-identifiers>

[S8] XRPL, Credentials. <https://xrpl.org/docs/concepts/decentralized-storage/credentials>

[S9] W3C, Decentralized Identifiers. <https://www.w3.org/TR/did/>

[S10] XRPL, LoanPay. <https://xrpl.org/docs/references/protocol/transactions/types/loanpay>

[S11] Rawlings, Mayne et Diehl, Model Predictive Control Theory Computation and Design. <https://sites.chemengr.ucsb.edu/~jbrow/mpc/>

[S12] Rockafellar et Uryasev, travaux sur l'optimisation de la CVaR. <https://sites.math.washington.edu/~rtr/papers.html>

[S13] Model Context Protocol, Security Best Practices. https://modelcontextprotocol.io/docs/2025-11-25/tutorials/security/security_best_practices

[S14] Stripe, Identity. <https://docs.stripe.com/identity>

[S15] Pencil, site et redirection pen.dev. <https://www.pencil.dev/>

[S16] Expo, Introduction to Expo Router. <https://docs.expo.dev/router/introduction/>

[S17] Union européenne, RGPD 2016/679. <https://eur-lex.europa.eu/eli/reg/2016/679/oj/eng>

[S18] Union européenne, MiCA 2023/1114. <https://eur-lex.europa.eu/eli/reg/2023/1114/oj/eng>

[S19] Union européenne, règlement IA 2024/1689. <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32024R1689>

[B1] FlowCredit_CDC_Complet_v1.1.zip, PDF de 20 pages fourni par l'équipe ; base de migration décrite au chapitre 3.